

```
In [ ]: import marimo as mo
import pandas as pd
from ortools.sat.python import cp_model

tam = 6

DIAS = ['Seg', 'Ter', 'Qua', 'Qui', 'Sex']
PERIODOS = range(1, tam)
```

Gerador de horários escolares (CP-SAT)

Este notebook cria e valida horários escolares usando **programação por restrições** com o solver CP-SAT da biblioteca `ortools`.

Dados de entrada

Os dados estão em ficheiros CSV, lidos com `pandas` (`pd.read_csv(...)`):

Ficheiro	Conteúdo
<code>turmas.csv</code>	lista de turmas
<code>disciplinas.csv</code>	disciplina, professor, carga semanal, se é de duplo período, sala especial
<code>disponibilidade_excecoes.csv</code>	tempos em que cada professor não pode dar aulas
<code>salas.csv</code>	tipos de sala e quantidade disponível de cada tipo

A leitura com `pd.read_csv` foi feita com ajuda de um LLM.

Ideia do modelo

Para cada combinação (turma, disciplina, dia, período) existe uma variável booleana `x[t, d, dia, p]`, que vale 1 se a turma `t` tem a disciplina `d` nesse tempo. As regras R1–R7 são restrições sobre estas variáveis, e o solver procura uma atribuição que as cumpra todas.

Estrutura do notebook

1. `criaHorarioEdaPrint`: constrói o modelo, resolve-o e imprime os horários.
2. `CheckHorario` e funções auxiliares: verificam se um horário cumpre as regras e medem a diferença entre dois horários.

```
In [ ]: def criaHorarioEdaPrint(pasta, H0):
    turmas = pd.read_csv(f'{pasta}/turmas.csv')['turma'].tolist()
```

```

disciplinas = pd.read_csv(f'{pasta}/disciplinas.csv')
disponibilidades = pd.read_csv(f'{pasta}/disponibilidade_excecoes.csv')
salas = pd.read_csv(f'{pasta}/salas.csv')

DISCS = disciplinas['disciplina'].tolist()

model = cp_model.CpModel()

# x[t,d,dia,p] = 1 se a turma t tem a disciplina d nesse tempo
x = {(t, d, dia, p): model.NewBoolVar(f'x_{t}_{d}_{dia}_{p}')}
    for t in turmas for d in DISCS for dia in DIAS for p in PERIODOS}

# R1: em cada tempo, no máximo uma disciplina por turma
for t in turmas:
    for dia in DIAS:
        for p in PERIODOS:
            model.AddAtMostOne(x[t, d, dia, p] for d in DISCS) # ele percorre
as disciplinas e adiciona apenas 1, ou seja para cada dia periodo apenas existe 1
disciplina

#R2: Cada disciplina cumpre exatamente a carga semanal definida em
'disciplinas.csv', para cada turma.
carga = disciplinas.set_index('disciplina')['carga_semanal'] # esta linha põe
a disciplina como rotulo para as colunas e seleciona apenas a carga semanal
criando um set

    for t in turmas:
        for d in DISCS:
            model.Add(
                sum(x[t, d, dia, p] for dia in DIAS for p in PERIODOS) == carga[d]
# faz com que ao fim da semana a aula ocoore o numero de vezes que a disciplina
precisa
            )

#R3 No máximo uma aula da mesma disciplina por dia, por turma – exceto disciplinas
de duplo período (ver R4), em que o bloco de 2 tempos conta como uma só ocorrência
nesse dia.

duploPeriodos = disciplinas.set_index('disciplina')['duplo_periodo']

for t in turmas:
    for d in DISCS:
        for dia in DIAS:
            if duploPeriodos[d] == 'nao':
                model.Add(sum(x[t, d, dia, p] for p in PERIODOS) = 1) #
garante que se a disciplina nao for de duplo periodos num dia d apenas ocoore 1
vez

            else:
                model.Add(sum(x[t,d,dia,p] for p in PERIODOS = 2 ) # garante
que se a disciplina nao for de duplo periodos num dia d apenas ocoore 2 vezes mas
nao garante que sejam seguidos

#R4 Disciplinas marcadas `duplo_periodo=sim` só podem ser dadas em blocos de 2
tempos consecutivos, no mesmo dia (nunca um tempo isolado).

```

```

for t in turmas:
    for d in DISCS:
        if duploPeriodos[d] == 'sim':
            for dia in DIAS:
                # 1. no máximo 2 tempos por dia
                model.Add(sum(x[t, d, dia, p] for p in PERIODOS) <= 2)

                # 2. nenhum tempo fica sozinho: se há aula em p, há também em
                # p-1 ou p+1

                for p in PERIODOS:
                    vizinhos = []
                    if p > 1:
                        vizinhos.append(x[t, d, dia, p - 1])
                    if p < 5:
                        vizinhos.append(x[t, d, dia, p + 1])
                    model.Add(x[t, d, dia, p] <= sum(vizinhos))

#R5 Um professor não pode dar duas aulas em simultâneo, mesmo que sejam a turmas
ou disciplinas diferentes.
discPorProf = disciplinas.groupby('professor')['disciplina'].apply(list) #
cria uma lista por professor que diz as disciplinas dele

for professor, disc in discPorProf.items():
    for dia in DIAS:
        for p in PERIODOS:
            model.AddAtMostOne(x[t, d, dia, p] for t in turmas for d in disc)

#R6. Um professor só pode dar aulas nos tempos em que está disponível
(`disponibilidade_excecoes.csv`).
for professor, dispo in disponibilidades.groupby('professor'):
    ds = discPorProf.get(professor, []) # cria uma lista por professor e se um
professor nao tiver disciplinas retorna uma lista vazia em vez de dar erro

    for _, r in dispo.iterrows(): # o iterrows percorre um DataFrame linha a
linha associando um indice á linha
        model.Add(sum(x[t, d, r['dia'], r['periodo']] for t in turmas for d in
ds) == 0)

#R7 Cada aula ocupa uma sala. Disciplinas com `sala_especial` só podem usar salas
desse tipo; as restantes usam salas `normal`. Em nenhum tempo o número de aulas a
decorrer num tipo de sala pode exceder a `quantidade` desse tipo definida em
`salas.csv`.
tipoDeSala = disciplinas.set_index('disciplina')['sala_especial'].fillna('Sala
Normal')

for _, s in salas.iterrows():
    ds = tipoDeSala[tipoDeSala == s['sala']].index # Para cada tipo de sala,
ds = nomes das disciplinas que usam esse tipo de sala

    for dia in DIAS:
        for p in PERIODOS:
            model.Add(
                sum(x[t, d, dia, p] for t in turmas for d in ds) <=
s['quantidade'] # garante que sao usadas apenas uma quantidade menor ou igual
dessa sala num periodo
            )

```

#01Minimizar o número total de "buracos" no horário de cada professor – um buraco é um tempo livre, no meio do dia, entre a primeira e a última aula desse professor nesse dia.

```
buracos = []
for professor, ds in discPorProf.items(): # .items() devolve pares (índice,
valor), um por linha da Series
    for dia in DIAS:
        atual, anterior, depois = {}, {}, {}
        for p in PERIODOS:
            atual[p] = model.NewBoolVar(f'atual_{professor}_{dia}_{p}')
            anterior[p] = model.NewBoolVar(f'anterior_{professor}_{dia}_{p}')
            depois[p] = model.NewBoolVar(f'depois_{professor}_{dia}_{p}')

            model.Add(atual[p] == sum(x[t, d, dia, p] for t in turmas for d in
ds))

        for p in PERIODOS:
            model.Add(anterior[p] >= atual[p])
            if p > min(PERIODOS):
                model.Add(anterior[p] >= anterior[p - 1])

            model.Add(depois[p] >= atual[p])
            if p < max(PERIODOS):
                model.Add(depois[p] >= depois[p + 1])

            buraco = model.NewBoolVar(f'gap_{professor}_{dia}_{p}')
            model.Add(buraco >= anterior[p] + depois[p] - 1 - atual[p])
            buracos.append(buraco)

if H0 is None:
    #Nao existe horario predefinido so intressa minimizar os buracos
    model.Minimize(sum(buracos))
else:
    # já existe um horário muda-lo o minimo
    for (t, d, dia, p), var in x.items():
        model.AddHint(var, 1 if H0.get((t, dia, p)) == d else 0)

mantidas = [x[t, d, dia, p]
              for (t, dia, p), d in H0.items()
              if (t, d, dia, p) in x]
model.Maximize(100 * sum(mantidas) - sum(buracos))

solver = cp_model.CpSolver()
estado = solver.Solve(model)

if estado in (cp_model.OPTIMAL, cp_model.FEASIBLE):
    print(f"Estado: {solver.StatusName(estado)} | Buracos totais:
{int(solver.ObjectiveValue())}\n")

prof_de = disciplinas.set_index('disciplina')['professor'].to_dict()
```

```

professores = disciplinas['professor'].unique()

# tabelas vazias: linhas = períodos, colunas = dias
horario_turma = {t: pd.DataFrame('', index=PERIODOS, columns=DIAS) for t
in turmas}
horario_prof = {p: pd.DataFrame('', index=PERIODOS, columns=DIAS) for p in
professores}

for (t, d, dia, p), var in x.items():
    if solver.Value(var):
        horario_turma[t].loc[p, dia] = f'{d} ({prof_de[d]})'
        horario_prof[prof_de[d]].loc[p, dia] = f'{d} ({t})'

if H0 is None:
    print("=" * 30, "HORÁRIOS DAS TURMAS", "=" * 30)
else:
    print("=" * 30, "HORÁRIOS DAS TURMAS V2", "=" * 30)

for t, df in horario_turma.items():
    df.index.name = 'Tempo'
    print(f"\nTurma {t}")
    print(df.to_string())

if H0 is None:
    print("=" * 28, "HORÁRIOS DOS PROFESSORES V2", "=" * 28)
else:
    print("=" * 28, "HORÁRIOS DOS PROFESSORES V2", "=" * 28)

for prof, df in horario_prof.items():
    df.index.name = 'Tempo'
    print(f"\nProfessor {prof}")
    print(df.to_string())
if estado not in (cp_model.OPTIMAL, cp_model.FEASIBLE):
    print(f'[{pasta}] Não foi encontrada solução:', solver.StatusName(estado))
    return None

print(f"Estado: {solver.StatusName(estado)} | Buracos totais:
{int(solver.ObjectiveValue())}\n")

horario = {}
for (t, d, dia, p), v in x.items():
    if solver.Value(v):
        horario[(t, dia, p)] = d
return horario

```

criaHorarioEdaPrint(pasta, H0)

Cria um horário (ou o adapta) e imprime-o.

Parâmetros

- `pasta` : pasta onde estão os ficheiros de dados.
- `H0` : horário anterior, ou `None` .

- `H0 = None` : cria o horário **do zero** e minimiza apenas os buracos dos professores.
- `H0 = horário` : cria uma **nova versão** para os novos dados, mantendo o mais possível do horário original. O horário antigo é usado como `AddHint` (ponto de partida para o solver) e o objetivo passa a ser maximizar as aulas iguais às de `H0` , com os buracos como critério secundário.

Formato do horário: dicionário `{(turma, dia, período): disciplina}` .

Restrições (obrigatórias)

Regra	O que garante
R1	Em cada tempo, uma turma tem no máximo uma disciplina.
R2	Cada disciplina cumpre exatamente a sua carga semanal, em cada turma.
R3	No máximo uma aula da mesma disciplina por dia e turma (disciplinas de duplo período podem ter 2 tempos, que contam como uma só aula).
R4	Disciplinas com <code>duplo_periodo = sim</code> só aparecem em blocos de 2 tempos consecutivos. Cada tempo tem de ter um vizinho com a mesma disciplina.
R5	Um professor não dá duas aulas em simultâneo, mesmo a turmas ou disciplinas diferentes.
R6	Um professor só dá aulas nos tempos em que está disponível.
R7	Cada aula ocupa uma sala do tipo certo (<code>sala_especial</code> ou normal), sem exceder a <code>quantidade</code> desse tipo em nenhum tempo.

Objetivo (otimização)

Um **buraco** é um tempo livre de um professor, no meio do dia, entre a sua primeira e a sua última aula. Para o detetar, para cada professor, dia e período usam-se 3 variáveis:

- `atual[p]` : o professor tem aula em `p` ;
- `anterior[p]` : o professor tem alguma aula em `p` ou antes;
- `depois[p]` : o professor tem alguma aula em `p` ou depois.

Há buraco em `p` quando `anterior[p]` e `depois[p]` são verdadeiros mas `atual[p]` é falso.

- **O1** (sem `H0`): minimizar o total de buracos.
- **Com `H0`** : maximizar `100 × (aulas mantidas) – buracos` . O peso 100 dá prioridade a alterar o mínimo possível e deixa os buracos como desempate.

Resolução e resultado

O solver corre no máximo **60 segundos**. Se encontrar solução (ótima ou apenas admissível), imprime os horários de cada turma e de cada professor (linhas = tempos, colunas = dias) e

devolve o horário como dicionário. Caso contrário, imprime o estado devolvido pelo solver (por exemplo `INFEASIBLE`).

```
In [ ]: def CheckHorario(horario,pasta):
    turmas = pd.read_csv(f'{pasta}/turmas.csv')['turma'].tolist()
    disciplinas = pd.read_csv(f'{pasta}/disciplinas.csv')
    disponibilidades = pd.read_csv(f'{pasta}/disponibilidade_excecoes.csv')
    salas = pd.read_csv(f'{pasta}/salas.csv')

    DISCS = disciplinas['disciplina'].tolist()

    if not isinstance(horario, dict):
        return False

    if not(checkR2(horario,turmas,disciplinas)):
        print("A condição R2 falhou")
        return False

    if not(checkR3(horario, turmas, disciplinas)):
        print("A condição R3 falhou")
        return False

    if not(checkR4(horario, turmas, disciplinas)):
        print("A condição R4 falhou")
        return False

    if not(checkR5(horario, turmas, disciplinas)):
        print("A condição R5 falhou")
        return False

    if not(checkR6(horario, turmas, disciplinas , disponibilidades)):
        print("A condição R6 falhou")
        return False

    if not(checkR7(horario, turmas, disciplinas , salas)):
        print("A condição R7 falhou")
        return False

    print(f'[{pasta}] Passou os testes todos:')
    return True

# A regra 1 é impossível de ser criada pois o horario esta guardado como um
# dicionario em que a chave [t, dia , p] equivale a uma unica disciplina Logo uma
# turma t num dia num periodo p so consegue ter uma disciplina

def checkR2(horario, turmas , disciplinas):
    for t in turmas:
        for index, d in disciplinas.iterrows():
            count = 0
            for dia in DIAS:
                for p in PERIODOS:
                    if horario.get((t, dia, p)) == d['disciplina']:
                        count += 1
            if count != int(d['carga_semanal']):
                return False
```

```

    return True

def checkR3(horario, turmas, disciplinas):
    for t in turmas:
        for index, d in disciplinas.iterrows():
            for dia in DIAS:
                count = 0
                for p in PERIODOS:
                    if horario.get((t, dia, p)) == d['disciplina']:
                        count += 1
                if count > 0:
                    if (count != 2 and d['duplo_periodo'] == 'sim'):
                        return False
                    elif (count != 1 and d['duplo_periodo'] == 'nao'):
                        return False
            return True

def checkR4(horario, turmas, disciplinas):

    discComDuplo = disciplinas.loc[disciplinas['duplo_periodo'] == 'sim',
    'disciplina']

    for t in turmas:
        for d in discComDuplo:
            for dia in DIAS:
                ps = []
                for p in PERIODOS:
                    if horario.get((t, dia, p)) == d:
                        ps.append(p)
                if ps and ps != [ps[0], ps[0] + 1]:
                    return False
            return True

def checkR5(horario, turmas, disciplinas):

    discDeProf = disciplinas.set_index('disciplina')['professor']
    for dia in DIAS:
        for p in PERIODOS:
            professoresADarAula = []
            for t in turmas:
                aula = horario.get((t, dia, p))
                if aula is not None:
                    prof = discDeProf[aula]
                    if prof in professoresADarAula:
                        return False
                    else:
                        professoresADarAula.append(prof)
            return True

def checkR6(horario, turmas, disciplinas, disponibilidades):

    discDeProf = disciplinas.set_index('disciplina')['professor']
    for dia in DIAS:
        for p in PERIODOS:
            for t in turmas:

```



```

        aula = horario.get((t, dia, p))
        if aula is not None:
            prof = discDeProf[aula]
            conflito = ((disponibilidades['professor'] == prof) &
                        (disponibilidades['dia'] == dia) &
                        (disponibilidades['periodo'] == p)).any()

            if conflito:
                return False

        return True

def checkR7(horario, turmas, disciplinas, salas):

    salaNormal = salas.loc[salas['tipo'] == 'normal', 'sala'].iloc[0]
    salaEspecial = disciplinas.set_index('disciplina')
    ['sala_especial'].fillna(salaNormal)

    for dia in DIAS:
        for p in PERIODOS:
            for index, s in salas.iterrows():
                count = 0
                for t in turmas:
                    aula = horario.get((t, dia, p))
                    if aula is not None and salaEspecial[aula] == s['sala']:
                        count += 1

                if count > s['quantidade']:
                    return False

    return True

def diferencaHorarios(h1, h2):
    diferencas = 0
    for chave in h1:
        if h1[chave] != h2.get(chave):
            diferencas += 1
    for chave in h2:
        if chave not in h1:
            diferencas += 1
    print('As diferenças entre os 2 horarios é ' + str(diferencas))
    return diferencas

```

Verificação de horários

Estas funções são **independentes do solver**. Recebem um horário já pronto e confirmam, regra a regra, que é válido para os dados de uma pasta.

- **CheckHorario(horario, pasta)** : lê os CSV e chama **checkR2** a **checkR7** . Pára na primeira regra que falhar e diz qual foi. Devolve **True** se o horário for válido.
- **checkR2** a **checkR7** : cada uma verifica uma regra, com a mesma numeração do modelo. A **R1** não precisa de verificação, porque o horário é um dicionário com chave

(turma, dia, período) e por isso cada turma só pode ter uma disciplina por tempo.

- **diferencaHorarios(h1, h2)** : conta em quantos tempos os dois horários diferem (aulas diferentes, ou presentes num horário e ausentes no outro). Serve para medir quanto mudou entre a versão original e a versão adaptada.

Como é usado abaixo:

1. **H0** é validado com os dados em **dados** .
2. **H1** (a versão adaptada) é validado com os dados em **dados_v2** .
3. Calcula-se a diferença entre **H0** e **H1** , que deve ser pequena, já que **H1** foi obtido maximizando as aulas mantidas.

```
In [ ]: H0 = criaHorarioEdaPrint('dados' , None)
H02 = criaHorarioEdaPrint('dados_v2' , None)
H1 = criaHorarioEdaPrint('01_basico' , None)
H2 = criaHorarioEdaPrint('02_medio' , None)
H3 = criaHorarioEdaPrint('03_grande' , None)
H4 = criaHorarioEdaPrint('04_inviavel_laboratorio' , None)
H5 = criaHorarioEdaPrint('05_inviavel_professor' , None)
H6 = criaHorarioEdaPrint('06_casos_limite' , None)
H7 = criaHorarioEdaPrint('07_turma_lotada' , None)

CheckHorario(H1 , '01_basico')
CheckHorario(H2 , '02_medio')
CheckHorario(H3 , '03_grande')
CheckHorario(H4 , '04_inviavel_laboratorio')
CheckHorario(H5 , '05_inviavel_professor')
CheckHorario(H6 , '06_casos_limite')
CheckHorario(H7 , '07_turma_lotada')
CheckHorario(H0 , 'dados')
CheckHorario(H02 , 'dados_v2')
```

1. [Partilha 1](#)
2. [Partilha 2](#)
3. [Partilha 3](#)
4. [Partilha 4](#)
5. [Partilha 5](#)
6. [Partilha 6](#)
7. [Partilha 7](#)
8. [Partilha 8](#)